Portable DS18B20 driver

Alireza Khajehvandi

STM32F107VCT6 / PB10

Implementation and learning guide

2026-10-10

Alireza Khajehvandi online

Portfolio · GitHub · LinkedIn

YouTube · Telegram · Instagram

Contents

1 Goal, boundaries and starting point **4**

1.1 Hardware information versus verified measurements 4

1.2 What is delivered 4

2 Connect and understand the sensor **5**

2.1 Three-wire connection 5

2.2 Power and startup 5

2.3 Debug interface 5

3 Architecture and portability **6**

3.1 The hardware contract 6

3.2 Ownership and concurrency 7

4 Functions and practical examples **7**

4.1 Start with the supplied example 7

4.2 Initialize the bus: OneWire_Init 7

4.3 Obtain an address: ReadRom and IsValidRom 8

4.4 Read one complete measurement: ReadTemperatureBlocking 8

4.5 Split conversion from reading 9

4.6 Prepare buffers: ArrayInit 9

4.7 Service the array: ArrayProcess 10

4.8 Replace a chosen slot: ArrayReplace 11

4.9 Configure and verify resolution 11

4.10 Raw helpers and errors 12

5 Stable slots and replacement **12**

5.1 An unambiguous replacement . . . 12

5.2 Ambiguity and manual control . . . 13

5.3 Incomplete scans and capacity . . . 13

5.4 Persistence is an application decision . . . 13

6 Timing, efficiency and error handling **13**

6.1 Real execution costs . . . 14

6.2 Validate before consuming . . . 14

6.3 Common failure diagnoses . . . 14

7 Build, program and regenerate **15**

7.1 PlatformIO . . . 15

7.2 Keil . . . 15

7.3 CubeMX regeneration without losing the library . . . 15

7.4 Use the next-project template . . . 16

8 Validation and the board acceptance procedure **16**

8.1 Automated evidence . . . 16

8.2 Board test steps . . . 17

8.3 Maintaining evidence . . . 17

1 Goal, boundaries and starting point

I organize each sensor or IC as an independent project. Its folder contains all application sources, portable driver sources, vendor MCU support, CubeMX settings, Keil configuration and PlatformIO configuration. Copying a project does not require the repository root or any sibling project. This first project uses DS18B20 on PB10 and STM32F107VCT6; the library itself does not depend on that processor.

The original copied application included LCD, relays, electrical measurement, multiple temperature buses, Flash settings and build artifacts. Those functions are removed from the new project. Only the development foundation and reviewed portable protocol routines are retained. The raw template contains no sensor code.

1.1 Hardware information versus verified measurements

The copied configuration specifies an 8 MHz HSE crystal, PLL multiplier nine, 72 MHz core, 36 MHz APB1, 72 MHz APB2 and a 3.3 V design supply. The MCU has 256 KiB Flash and 64 KiB RAM[1]. The user identifies PB10 as the connected sensor pin. These facts define the firmware assumptions; I have not measured the physical board, supply or waveform. Old PC7, relay and LCD connections are not used.

1.2 What is delivered

The project supports one addressed sensor or a caller-sized address array with corresponding temperature and status arrays. Conversion scheduling is cooperative, ROM discovery is incremental, and replacement preserves a slot when identity is unambiguous. The portable implementation uses caller-owned static storage and no heap. External sensor power is required. Parasite power, persistent mapping and hardware-assisted transfer are future extensions, not implemented features.

2 Connect and understand the sensor

The original datasheet[2] defines a unique eight-byte ROM and a nine-byte scratchpad. ROM byte zero is family code 0x28, bytes one through six identify the device, and byte seven is CRC8. The first two scratchpad bytes form a signed temperature in units of 1/16 degree Celsius. Byte four selects resolution; lower-resolution undefined bits must be masked. The final byte protects the scratchpad with CRC8. Resolution controls granularity and conversion time, not the sensor's absolute accuracy.

2.1 Three-wire connection

Connect DQ to PB10, VDD to 3.3 V and GND to the MCU ground. A typical 4.7 kOhm resistor connects DQ to 3.3 V. Confirm the actual package pinout in the original datasheet; do not infer it from cable colours. A long or heavily loaded bus may require an adjusted pull-up and topology after waveform measurements. PB10 is open-drain, initially released high, with no internal pull-up. Writing a GPIO one releases the line; it must never drive against a sensor pulling the line low.

2.2 Power and startup

This adapter supplies no strong pull-up and therefore does not support parasite power. Use the VDD connection. The sensor's power-on temperature register contains 85 C; that same temperature can also be a real measurement. The array starts conversion before reading and never rejects 85 C merely because of its value. Direct scratchpad reads are the caller's responsibility: start and complete a conversion first. If the sensor resets during a conversion, value alone cannot prove freshness; systems requiring stronger detection need additional diagnostics.

2.3 Debug interface

Connect PA13/SWDIO, PA14/SWCLK, common ground and the probe's target voltage reference. Supply the target from its intended board supply. Keep SWD enabled; JTAG is released by the MSP setup. A probe connection is separate from the temperature bus and does not provide evidence that sensor wiring is correct.

3 Architecture and portability

Location	Responsibility
Core	CubeMX-owned main, interrupts, HAL configuration and system clock.
Application	Board adapter, DWT delay, GPIO callbacks and example arrays.
Libraries/OneWire	Reset, bit slots, bytes, Dallas CRC8 and ROM search.
Libraries/DS18B20	Addressed commands, scratchpad decoding and array scheduler.
Drivers	Local vendor HAL and CMSIS shared by both build systems[3].
MDK-ARM	Keil project and Arm assembly startup.
Core/Startup	GNU assembly startup for PlatformIO.
tools and tests	Regeneration synchronization, checks and emulated C tests.

3.1 The hardware contract

`OneWire_Bus` binds a context pointer and callbacks. Drive-low asserts the line; release makes it high impedance; read samples the pin. Delay accepts microseconds and must run while interrupts are disabled. The optional critical callbacks must be supplied as a pair: entering returns previous interrupt state, leaving restores that exact state. Supplying neither requires the application's environment to guarantee bit timing independently.

On this Cortex-M3, DWT cycle counting supplies the delay. The adapter enables trace and cycle counting after the system clock is configured. GPIO BSRR/BRR registers perform single writes without GPIO reconfiguration. The elapsed-cycle subtraction handles counter wrap for these short delays. Other MCUs, including ARM families without a usable DWT counter, should implement a timer callback. The portable headers include only standard C headers and other portable headers.

3.2 Ownership and concurrency

The caller allocates the array object, ROM array, discovery buffer, temperatures and per-slot status. Both ROM buffers have the configured capacity, are separate, and remain alive throughout use. No allocation or global library singleton is required. Different buses may have separate instances. Calls on the same bus must be serialized by the application; never invoke the driver from an ISR or change core frequency during a transaction. An RTOS mutex belongs around complete transactions, not around an individual callback.

4 Functions and practical examples

4.1 Start with the supplied example

Call `App_Init()` once after HAL, clock and GPIO initialization, then call `App_Process()` repeatedly from the main loop. Set `APP_SENSOR_COUNT` to one for one sensor or to the required capacity. `g_sensor_roms[i]`, `g_temperature_c[i]` and `g_temperature_status[i]` refer to the same slot. Index zero is the first sensor. Only an OK status permits using that slot's temperature.

For another MCU, bind its GPIO/time callbacks first. The library offers a complete blocking measurement, separate start/read operations and a cooperative array interface. The array interface also works with a single sensor.

4.2 Initialize the bus: OneWire_Init

Inputs are the destination bus and a callback configuration. The function copies a valid configuration, releases the line and returns true; it does not discover a sensor. The mandatory callbacks and paired critical callbacks were introduced in the architecture section. The concrete STM32 example is in `Application/Src/app.c`.

```
1 OneWire_Bus bus;
2 /* callbacks contains your board's GPIO/time functions. */
3 bool bus_ok = OneWire_Init(&bus, &callbacks);
```

Do not use the bus if initialization fails. Keep it alive while the array refers to it. The following

snippets assume initialization succeeded.

4.3 Obtain an address: ReadRom and IsValidRom

`DS18B20_ReadRom` takes the bus and an eight-byte output buffer and returns a status. Use it only when exactly one sensor is physically connected. `DS18B20_IsValidRom` checks family and CRC of a supplied address; it does not prove the sensor is currently present.

```
uint8_t rom[DS18B20_ROM_SIZE];
DS18B20_Status result = DS18B20_ReadRom(&bus, rom);
bool address_ok = (result == DS18B20_STATUS_OK);
```

For several sensors, choose the array's automatic discovery or use `OneWire_SearchFirst` and `OneWire_SearchNext` with caller-owned search state. Each successful search supplies a CRC-valid ROM; also check its DS18B20 family. Search order does not identify a physical location.

4.4 Read one complete measurement: ReadTemperatureBlocking

Inputs are bus, ROM, nominal polling budget in milliseconds and output pointer. The function starts conversion, waits and reads a validated temperature. Only OK updates the output; on failure it stays unchanged. Use 1000 ms for an unknown resolution. Bus overhead adds to the nominal budget.

```
float temperature_c = 0.0f;
if (address_ok) {
    result = DS18B20_ReadTemperatureBlocking(
        &bus, rom, 1000U, &temperature_c);
}
bool temperature_valid = (result == DS18B20_STATUS_OK);
```

This continues the preceding example. Use the output only when valid; initial zero and old values are not successful measurements. This function can block for roughly a conversion interval. For concurrent main-loop work use the array interface or the split sequence below.

4.5 Split conversion from reading

`DS18B20_StartConversion` sends Convert T to a ROM; null broadcasts to the bus. OK means command sent, not measurement completed. `DS18B20_IsConversionReady` checks the conversion busy bit. `DS18B20_ReadTemperature` reads a completed conversion and performs CRC and range checks; it does not start a new measurement. Its output changes only on success.

On STM32 use the HAL tick; elsewhere supply a monotonic millisecond clock. Start once and retain the state variables between main-loop iterations:

```
bool waiting = false;
uint32_t deadline = 0;
temperature_valid = false;
result = DS18B20_StartConversion(&bus, rom);
if (result == DS18B20_STATUS_OK) {
    deadline = HAL_GetTick() + 760U; /* After command. */
    waiting = true;
}
```

Run this fragment repeatedly in the main loop:

```
if (waiting &&
    (int32_t)(HAL_GetTick() - deadline) >= 0) {
    waiting = false;
    result = DS18B20_IsConversionReady(&bus)
        ? DS18B20_ReadTemperature(&bus, rom, &temperature_c)
        : DS18B20_STATUS_TIMEOUT;
    temperature_valid = (result == DS18B20_STATUS_OK);
}
/* Other main-loop work continues while waiting. */
```

Do not reset/search or send another bus command between conversion and polling. The array scheduler handles this sequence for you and is usually simpler.

4.6 Prepare buffers: ArrayInit

Inputs are manager, initialized bus, mutable ROM array, separate discovery buffer, temperature array, status array and count. All buffers have that capacity and remain alive while sampling. Invalid pointers/count, duplicate ROMs and invalid nonzero ROMs cause false. Zero ROMs request automatic discovery.

```c
#define N 4U /* Use 1U for one sensor. */
static DS18B20_Array sensors;
static DS18B20_Rom addresses[N] = {{0}};
static DS18B20_Rom discovered[N];
static float temperatures[N];
static DS18B20_Status status[N];
bool initialized = DS18B20_ArrayInit(
    &sensors, &bus, addresses, discovered,
    temperatures, status, N);
if (initialized) sensors.sample_interval_ms = 250U;
```

Run once, not every loop iteration. The interval is the idle gap after a cycle, not conversion duration. For known ROMs fill the address array before initialization in wire order: family, six serial bytes, CRC. Discovery/replacement can change this array, so do not declare it const. Static buffers avoid large stack use.

4.7 Service the array: ArrayProcess

Inputs are manager and current millisecond tick. Call frequently. True means a complete cycle ended, even if some slots failed; false means no completed cycle yet. Check each slot's status separately. Index `i` always corresponds to the same address-array index.

```c
static float latest_c[N];
static bool valid[N];
if (initialized &&
    DS18B20_ArrayProcess(&sensors, HAL_GetTick())) {
    for (size_t i = 0; i < N; ++i) {
        valid[i] = (status[i] == DS18B20_STATUS_OK);
        if (valid[i]) latest_c[i] = temperatures[i];
    }
}
```

This optional snapshot preserves the last numbers but marks failed slots invalid. Use a value for display/control only when its valid flag is true. The library's own outputs become NAN during conversion or errors.

4.8 Replace a chosen slot: ArrayReplace

Inputs are manager, zero-based destination index and new valid ROM. True updates the mapping and invalidates the old temperature. False indicates a bad ROM/index, a ROM already used by another slot or a non-idle array. Obtain the new ROM from a complete scan or a separate single-sensor setup, not Read ROM on a multidrop bus.

```
/* replacement_rom was obtained and validated earlier. */
if (sensors.phase == 0U) {
    bool replaced = DS18B20_ArrayReplace(
        &sensors, 1U, replacement_rom); /* Second slot. */
    /* Check replaced, then wait for a valid new sample. */
}
```

Use at least two slots for this example and call from the same task as processing. ROM changes do not survive reset unless the application saves them. The following section explains when replacement can be automatic.

4.9 Configure and verify resolution

`DS18B20_WriteScratchpad` takes bus, ROM, high alarm, low alarm and resolution. The example sets 10-bit resolution and alarms at 30 and 10 C. Settings are volatile; no EEPROM copy is performed. A successful command alone is not an addressed sensor acknowledgment, so verify using `DS18B20_ReadScratchpad` and `DS18B20_GetResolution`.

```
result = DS18B20_WriteScratchpad(
    &bus, rom, 30, 10, DS18B20_RESOLUTION_10_BIT);
uint8_t pad[DS18B20_SCRATCHPAD_SIZE];
if (result == DS18B20_STATUS_OK)
    result = DS18B20_ReadScratchpad(&bus, rom, pad);
bool configured = (result == DS18B20_STATUS_OK) &&
    DS18B20_GetResolution(pad) == DS18B20_RESOLUTION_10_BIT &&
    pad[2] == 30U && pad[3] == 10U;
```

Configure before sampling or while idle. The array still uses its conservative 12-bit wait. `DS18B20_GetConversionTimeMs` returns 94, 188, 375 or 750 ms for 9–12 bits; add timing margin when designing your own scheduler.

4.10 Raw helpers and errors

`DS18B20_ReadScratchpad` reads and validates nine bytes.

After a successful read, `DS18B20_DecodeRaw` extracts the signed raw temperature and masks undefined bits.

`DS18B20_RawToCelsius` converts the raw value to degrees.

For normal measurements, use `ReadTemperature`. It also rejects values outside the sensor's valid range.

OK permits use; BUSY means wait; NO_DEVICE means no reset presence; CRC_ERROR means unusable data; TIMEOUT means conversion was not ready; OUT_OF_RANGE means an impossible value; INVALID_ARGUMENT means correct the call or ROM. A missing addressed sensor can produce CRC_ERROR if another sensor answers reset. Always check status, not just the numeric value. The OneWire reset/bit/byte functions are protocol primitives usually called by the sensor layer; their implementation remains in the source files rather than being copied into this report.

5 Stable slots and replacement

Search ROM returns device order according to bit-level arbitration, which is not a physical location identifier. Existing ROMs are matched by all eight bytes, so a changed search order does not rearrange outputs. With no missing assigned devices, unknown ROMs fill the first empty slots.

5.1 An unambiguous replacement

Suppose addresses are A, B and C. Disconnect B and connect D. A complete scan finds A, C and D: one assigned address is missing and one address is unknown. The mapping becomes A, D and C, preserving array index one. Discovery runs approximately every two seconds and sampling follows, so the change is not instantaneous. Temperature becomes NAN while busy or invalid.

5.2 Ambiguity and manual control

If B and C disappear while D and E appear, software cannot infer which physical location belongs to D. It keeps the old mappings rather than inventing a pairing. Replace sensors one at a time or call `DS18B20_ArrayReplace` for a chosen slot when phase is zero. The explicit function rejects a ROM already used by another slot. Phase zero occurs between cycles; make the call from the same task that services the array. No manual replacement is allowed during conversion.

5.3 Incomplete scans and capacity

CRC failure, impossible search-bit responses or missing presence reset the search state. Automatic mapping requires a complete enumeration. If more DS18B20 devices exist than the discovery buffer can hold, the overflow flag is raised and automatic mapping is inhibited. Increase the configured slot count to accommodate the entire bus. A physically present but electrically unreadable sensor remains an error; there is no way to identify its location without an address or a separate bus.

5.4 Persistence is an application decision

The ROM table is stored in RAM. It does not survive a processor reset unless the application reloads it. To retain labels, initialize with known ROMs or implement application-owned nonvolatile storage with CRC and power-loss safety. The library neither reserves Flash pages nor chooses a storage policy. Do not write Flash every sampling cycle. Each project can add storage independently.

6 Timing, efficiency and error handling

The scheduler has idle, discovery, convert-start, wait, read and completed phases. Each call advances one small step. Discovery performs one ROM search, reading performs one addressed scratchpad, and waiting returns immediately until the deadline. Broadcast conversion begins all externally powered sensors together. The 760 ms deadline includes command execution and millisecond rounding around the sensor's 750 ms maximum. Deadlines use signed subtraction and handle wrap; service gaps must remain below half the 32-bit millisecond range.

6.1 Real execution costs

Nonblocking conversion does not make the 1-Wire transport asynchronous. At standard speed, a ROM search uses approximately 15 ms and an addressed read approximately 12 ms of foreground time. A complete array cycle grows with sensor count. Interrupts can execute during recovery and the long conversion wait. Critical requested delay durations are about 70 us for reset sampling, 60 us for write zero, 6 us for write one and 13 us for read sampling. Callback and register overhead add to these figures; they are not measured upper bounds on this board. For stricter real-time requirements replace the transport with a hardware master or timer/DMA design and validate the actual waveform with an oscilloscope.

6.2 Validate before consuming

ROM and scratchpad CRC8 use the Dallas polynomial in reflected form 0x8C. Scratchpad configuration and fixed reserved bytes are checked to reject an all-zero line response, which would otherwise have a mathematically valid zero CRC. Signed raw decoding masks undefined bits for 9–11 bit resolution. Converted values outside -55 to 125 C are rejected. At conversion start, every array output becomes NAN and BUSY; success changes its status to OK. Missing, timeout, CRC and range failures retain NAN. Never compare NAN with equality; inspect status or use the standard `isnan` function.

6.3 Common failure diagnoses

No device: check power, ground, pull-up and PB10. CRC error: inspect supply noise, cable loading and timing; a disconnected addressed sensor can also cause CRC error when another sensor still answers reset. Timeout: check sensor power, conversion readiness and a line held low. Invalid argument: check the ROM CRC, buffer lifetime, count and duplicate addresses. Temperature 85 C: verify that a conversion completed and investigate supply resets, while accepting that 85 C may be real. A low idle line is rejected before reset; a line that remains low after the presence window is also rejected.

A configurable idle gap follows each cycle; `sample_interval_ms` starts at 250 ms and can be adjusted before servicing. This also limits retries when no sensor is attached. The conversion deadline is armed on the next call after command transmission, so an interrupt during the com-

mand cannot shorten it.

7 Build, program and regenerate

7.1 PlatformIO

Install PlatformIO, open the standalone folder and use the following commands. The project pins ST STM32 platform version 19.6.0 and builds local vendor sources directly. The board profile supplies MCU and probe settings; it does not mean your custom PCB is the evaluation board.

```
pio run
pio run -e stm32f107vc_stlink
pio run -t upload
pio run -e stm32f107vc_stlink -t upload
```

Both environments use the same sources and linker script. The GNU startup is local. Flash is 256 KiB and RAM is 64 KiB. First installation needs PlatformIO tool downloads; thereafter the project does not depend on sibling folders.

7.2 Keil

Open MDK-ARM/DS18B20.uvprojx. Install the STM32F1 device pack and Arm Compiler 6. Select the available SWD probe under Options for Target / Debug and Utilities. Set the STM32F10x 256 KiB Flash programming algorithm, Build, then Download. Watch the ROM, temperature and status arrays and the completed cycle counter. Project XML and source membership were checked; an actual Keil build was not performed because the compiler installation was not located in this environment.

7.3 CubeMX regeneration without losing the library

Open DS18B20.ioc, preserve the 8 MHz clock assumptions, PB10 open-drain configuration and Keep User Code. Libraries and Application are outside the generator's Core. The include and App calls are inside USER CODE blocks. CubeMX may rewrite the Keil project, so the configured post-generation script restores user source groups and include directories. If a relative script path is not resolved by your CubeMX version, select tools/post_generate.cmd explicitly in User

Actions. After every Generate, run:

```
python tools/sync_keil.py
python tools/check_project.py
pio run
```

The synchronization is idempotent, keeps the full Flash size, adds Arm startup, and discovers Application, Libraries and local HAL C sources. PlatformIO uses source wildcards; add the include directory of every new library manually. For a new peripheral, inspect HAL configuration, CubeMX interrupt handlers and MSP initialization too. Review the diff and build Keil. Real CubeMX regeneration has not been executed here; the structural checks document expected invariants.

7.4 Use the next-project template

Copy the entire raw template folder to a new project directory. It contains only the clock, SWD, startup, vendor support and empty App hooks. Rename the project using CubeMX when appropriate, rerun synchronization, add the next library and its pin settings, and create a local status file and bilingual reports. Keep each project's dependencies inside that folder.

8 Validation and the board acceptance procedure

8.1 Automated evidence

PlatformIO successfully builds the sensor example and raw template with warning checks enabled. Structural checks validate local file paths, Keil source membership including startup, protected App hooks, IOC pin settings and Flash size. The tests compile real portable C and execute it on an emulated Cortex-M3:

```
python -m pip install unicorn
python tests/run_tests.py
python tools/check_project.py
```

One test suite substitutes protocol responses to exercise array ordering, replacement, ambiguity, incomplete scans, overflow, CRC faults, missing sensors, timeout, signed decoding, range rejection and tick wrap. Another executes actual 1-Wire code against scripted line samples to check reset

failures, CRC vectors, ROM enumeration and requested critical delays. This is useful software evidence, not proof of sensor electronics or analog timing.

8.2 Board test steps

Build and program with the selected probe. Connect one externally powered sensor on PB10. Wait for at least one completed sample cycle; record ROM, status and a reasonable ambient temperature. Add sensors one at a time and verify each appears in the next free slot. Initialize known ROMs and confirm array order after restart. Disconnect one sensor: its status must fail and its temperature must be NAN. Connect a new sensor and verify it occupies the same index after discovery. Repeat with two simultaneous replacements and confirm mappings remain unchanged until manual resolution. Test an absent pull-up and a safely simulated stuck-low line. Inspect waveforms under the intended interrupt load and compare temperature against an independent reference at several points. Record results in the status file; physical tests have not yet been run.

8.3 Maintaining evidence

Update PROJECT_STATUS.md after each phase. Distinguish a source check, successful compiler build, actual CubeMX regeneration and physical measurement. Keep datasheet downloads outside the project; official links and a bibliography remain inside. Final report PDFs and editable LaTeX sources are retained in Git for GitHub publication. Rebuild and verify both PDFs after report edits. Downloaded fonts, rendering images and temporary build products remain excluded.

References

[1] STMicroelectronics, *STM32F107VC product documentation*, 2026, accessed 2026-10-10. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32f107vc.html

[2] Analog Devices / Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, 2019, rev. 6, accessed 2026-10-10. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[3] STMicroelectronics, *STM32CubeF1 official firmware sources*, 2026, vendor sources retain their individual licenses. [Online]. Available: https://github.com/STMicroelectronics/STM32CubeF1

About the author and contact

Alireza Khajehvandi.

Portfolio: https://alirezaaies.github.io/

GitHub: https://github.com/alirezaaies

LinkedIn: https://www.linkedin.com/in/alirezakhajehvandi

YouTube: https://www.youtube.com/@AlirezaAIES

Telegram: https://t.me/AlirezaAIES

Instagram: https://www.instagram.com/alireza.aies